

Brief - Proyecto GHL Sentinel (MVP Interno) — v2

Actualizado con confirmaciones del equipo de soporte técnico de HighLevel.

1. Objetivo

Desarrollar una prueba de concepto funcional de una plataforma de observabilidad para GoHighLevel, destinada inicialmente al uso interno de la agencia.

El sistema deberá monitorear el estado operativo de múltiples subcuentas y detectar anomalías antes de que sean reportadas por los clientes.

El objetivo no es construir un SaaS comercial desde el inicio, sino validar el valor operativo del concepto dentro de una agencia con una cantidad significativa de subcuentas.

2. Problema

Actualmente, la agencia administra decenas de clientes y múltiples subcuentas.

Cuando ocurre una falla (WhatsApp, Workflows, OAuth, Email, Phone), el primer indicador suele ser el reclamo del cliente.

GoHighLevel no ofrece una vista centralizada del estado operativo de todas las subcuentas ni una capa de observabilidad que permita detectar incidentes de forma proactiva.

3. Objetivo del MVP

Construir un dashboard interno capaz de responder, en tiempo real, cuatro preguntas fundamentales:

- ¿Cuántas subcuentas están completamente sanas?
- ¿Qué subcuentas presentan incidentes?
- ¿Qué servicio está afectado en cada caso?
- ¿Desde cuándo existe el problema?

4. Alcance inicial

El MVP incluirá los siguientes monitores, cada uno validado (con limitaciones conocidas) por el soporte técnico de HighLevel:

- Estado de Workflows.
- Estado de conexiones OAuth.
- Estado de WhatsApp.
- Estado de Email (LC Email) y Phone (LC Phone), como extensión menor si el tiempo lo permite.

No se implementarán funcionalidades comerciales, autenticación multiusuario ni facturación.

5. Detalle de detección por monitor (confirmado con soporte de HighLevel)

5.1 Workflows

Mecanismo: webhook al inicio y al final de cada workflow. Confirmado por HighLevel como uno de los workarounds más confiables disponibles hoy.

Limitación confirmada: no garantiza el 100%. Un workflow puede quedar colgado, un paso puede hacer timeout, o el flujo puede no llegar nunca al webhook final.

Feature derivada: en vez de reportar solo "falló", detectar el estado intermedio `Workflow Possibly Stuck` cuando pasó el tiempo esperado sin llegar al webhook final.

Riesgo de diseño a resolver: el timeout de "posiblemente colgado" **no puede ser un valor global**. Un workflow con wait steps intencionales (ej. seguimiento a 3 días) no puede compartir umbral con uno que debería resolverse en minutos. El timeout debe definirse **por workflow**, derivado de sus wait steps configurados o seteado manualmente al instrumentarlo.

Restricción operativa confirmada: no existe API para agregar la acción de webhook a un workflow de forma programática — es una acción manual dentro del Workflow Builder, workflow por workflow, subcuenta por subcuenta. Esto implica una tarea de configuración manual explícita antes de que el monitoreo esté activo, y debe dimensionarse como esfuerzo de setup, no asumirse como automatizable desde el día uno.

5.2 OAuth

Mecanismo confirmado: el status code HTTP no alcanza. Algunos proveedores devuelven `200 OK` aunque el token ya no sea válido (ej. Google Calendar devuelve `200` con `items: []` cuando antes siempre devolvía calendarios).

Arquitectura resultante:

```
HTTP Status
+
Validación de contenido
+
Frecuencia de actividad
```

Implicación de diseño: esto no es un monitor genérico de OAuth, sino **una estrategia distinta por proveedor** (Google Calendar, Facebook, etc.), ya que lo que cuenta como "respuesta sospechosa" depende del proveedor. Modelar como interfaz + implementación por proveedor (encaja con Clean Architecture / SOLID). Cada integración nueva que se quiera cubrir agrega un módulo, no reutiliza el existente.

5.3 WhatsApp

Limitación confirmada: hay latencia entre la desconexión real y su reflejo en el estado — un número puede aparecer como activo varios minutos después de desconectarse. No puede tratarse como monitoreo en tiempo real estricto.

Diseño resultante — estados intermedios en vez de binario:

-  Healthy
-  Suspected
-  Confirmed Offline

Parámetro a calibrar: ventana de tiempo entre `Suspected` y `Confirmed Offline`. Debe ser configurable, no una constante fija, hasta tener datos reales de cuánto tarda la señal en estabilizarse.

5.4 Email (LC Email)

Mecanismo confirmado: webhook oficial `LCEmailStats`, con datos de entregas, estadísticas y confirmaciones. Señal de salud nativa y confiable — el monitor de mayor certeza de los cuatro.

5.5 Phone (LC Phone)

Limitación confirmada: HighLevel confirma que no hay webhook events para LC Phone. No existe señal oficial.

Alternativa disponible:

```
Actividad
+
Llamadas recientes
+
Silencio
```

Parámetro a calibrar: el umbral de "silencio sospechoso" debe ser una **baseline por cliente**, no un valor absoluto — un negocio con 2 llamadas/día no puede compartir umbral con uno de 200/día. Este es, de los monitores incluidos, el de menor confiabilidad; debe comunicarse así internamente para no generar expectativas de precisión parejas entre monitores.

6. Health Score / Confidence Score

En lugar de un estado binario (Conectado / Desconectado), el sistema calculará un score de confianza por servicio, ya que ningún canal garantiza detección al 100%:

Workflow	95%
OAuth	100%
WhatsApp	82%
Email	100%
Phone	65%

Ejemplo de señales de entrada para WhatsApp:

```
Lista de números activos
+
Último mensaje enviado
+
Último mensaje recibido
+
Webhook recibido
+
Silencio
=
Health Score
```

Decisión de diseño para el MVP: el Confidence Score se implementa como una **capa de presentación** calculada sobre los estados discretos ya definidos en la sección 5 (Healthy/Suspected/Confirmed, pass/fail de validación de contenido, etc.), no como el motor de detección en sí. Esto evita que el criterio de éxito del MVP dependa de calibrar una fórmula de scoring sin datos históricos todavía. El score se afina una vez que el Event Store (sección 7) acumule observaciones reales.

Requisito de diseño: el score debe ser explicable — poder mostrar el desglose de señales que lo componen y su peso, no solo el número final. Un score que baja sin justificación visible genera más desconfianza que la que resuelve.

7. Event Store

En vez de guardar solo el estado actual ("WhatsApp = Healthy"), se guarda cada observación con su timestamp:

```
10:00 - WhatsApp OK
10:05 - WhatsApp OK
10:10 - WhatsApp sin actividad
10:15 - WhatsApp reconectado
```

Esto permite construir, a futuro: líneas de tiempo, métricas de uptime, MTTR (tiempo medio de recuperación), detección de patrones, e insumo para diagnóstico asistido por IA. Costo bajo al inicio (tabla append-only: timestamp / servicio / subcuenta / estado / señal cruda), y es la base de datos que alimenta el cálculo del Confidence Score.

Nota operativa: con ~80 subcuentas × varios monitores × observaciones frecuentes, definir una política de retención antes de pasar a producción (no es bloqueante para el MVP, pero sí para escalar).

8. Principios de diseño

- Arquitectura limpia (Clean Architecture).
- Principios SOLID.
- Desarrollo guiado por pruebas (TDD).
- Independencia de la infraestructura.
- Bajo acoplamiento entre módulos.
- Capacidad de reemplazar componentes sin afectar el dominio.

9. Arquitectura lógica



El Health Engine combina las señales de cada monitor (sección 5) para determinar el estado de salud de cada servicio, y expone tanto el estado discreto como el score calculado.

10. Stack propuesto

Componente	Tecnología	Justificación
Frontend	Next.js	Permite desarrollar tanto la futura landing como el dashboard en un único proyecto, con excelente soporte para SEO, routing y componentes reutilizables.
Backend	NestJS	Arquitectura modular, inyección de dependencias nativa y excelente alineación con Clean Architecture y SOLID.
ORM	Prisma	Independiza el dominio de la base de datos y facilita la migración entre SQLite y PostgreSQL.

Componente	Tecnología	Justificación
Base de datos (MVP)	SQLite	Simplicidad, rapidez y cero configuración durante la validación inicial.
Base de datos (Producción)	PostgreSQL	Escalabilidad y robustez una vez validado el producto.
Testing	Jest	Integración natural con NestJS y soporte sólido para TDD.

11. Riesgos y consideraciones operativas

- **Instrumentación manual de workflows:** no hay API para agregar acciones de webhook a workflows existentes; es trabajo manual por workflow y por subcuenta. Dimensionar como tarea de setup explícita.
- **Rate limits de la API de GHL:** 100 requests / 10 segundos por location. Relevante para cualquier monitor que dependa de polling (OAuth, Phone).
- **OAuth como N módulos, no uno:** cada proveedor integrado agrega una estrategia de validación propia; el costo de mantenimiento crece con cada integración nueva que se quiera cubrir.
- **Confiabilidad desapareja entre monitores:** Email y OAuth tienen señales fuertes; WhatsApp tiene latencia inherente; Phone no tiene señal oficial y depende de heurísticas de baseline por cliente. Comunicar esto internamente para calibrar expectativas.

12. Parámetros pendientes de calibración

Estos valores no deben quedar hardcodeados como constantes; deben ser configurables y ajustarse con datos reales de la operación:

1. Timeout de "posiblemente colgado" por workflow (sección 5.1).
2. Ventana de tiempo entre `Suspected` y `Confirmed Offline` para WhatsApp (sección 5.3).
3. Baseline de actividad/silencio esperado por cliente para Phone (sección 5.5).
4. Pesos de cada señal en el cálculo del Confidence Score (sección 6).

13. Criterio de éxito

El MVP se considerará exitoso cuando sea capaz de detectar y visualizar, utilizando datos reales de la operación de la agencia, incidentes en al menos uno de los servicios monitoreados antes de que sean detectados manualmente por el equipo de soporte.